



Universidad Autónoma de Baja California
Facultad de Ciencias Químicas e Ingeniería
ISTE

LPOO

Unidad I
Paradigma Orientado a Objetos

El paradigma orientado objetos a existido desde **inicios de 1960** pero no fue que hasta mediados de 1990 fue donde empezó a tener mucha popularidad a pesar de que ya existían lenguajes que manejaban el paradigma tales como (SmallTalk, C++, Python)



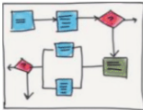
Todo paradigma consisten de dos elementos: Datos y Código, en donde unos programas se codifican en torno a "que esta pasando" o "a quien le esta afectando"

La primera forma de codificar se le conoce como **modelo orientado a procesos** este modelo *se caracteriza al programa en una serie de pasos de instrucciones lineales.*

Este modelo se puede interpretar como el código actúa a partir de datos.(paradigma estructurado) este tipo de modelo se utiliza mucho en lenguajes como C

Sin embargo una desventaja de usar dicho modelo, es que a medida que el proyecto crece, el código se hace más complejo y menos robusto, lo que provoca que el código sea mucho menos mantenible al paso del tiempo.

Para solucionar esa desventaja surgió la creación de un nuevo modelo llamado **Programación Orientada a Objetos(OOP)**, se caracteriza como datos que controlan el acceso al código a través de interfaces bien definidas de los datos (relaciones entre ellos).

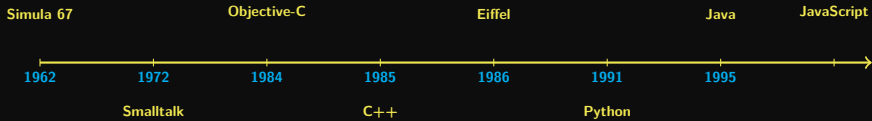
	Estructurado	POO
Programas	Datos y Funciones Separados	Datos y Funciones Unidos
Funciones	A base de datos	Interacciones de objetos
Diseño	Funciones y datos	Objetos
		

Todos lenguajes que utilizan el modelo de orientado a objetos proveen un mecanismo que ayuda a implementar las características de OOP, las cuales consisten en **abstracción, encapsulamiento, herencia, polimorfismo y composición**.

Estos son algunos lenguajes que proveen las herramientas para implementar el paradigma orientado a objetos.

- Java (JVM)
- C++
- C# (.NET)
- Python
- Ruby
- Javascript/TypeScript
- PHP

- Kotlin (JVM)
- Swift (iOS macOS)
- Scala (JVM)
- Dart (GOOGLE flutter)
- Lisp
- SmallTalk
- Groovy





Es un elemento esencial de la programación orientado a objetos. Los humanos resuelven la complejidad a través de la abstracción e de ahí donde proviene esta característica.

Por ejemplo cuando pensamos en un carro, no pensamos en las miles de partes individuales que contiene el carro, si no en un objeto con sus comportamientos y características únicas definido como carro.

De esta forma permite que un usuario convencional pueda usar un carro para ir a la tienda o a la casa, sin preocuparse de sus mil partes individuales.

Pueden ignorar los detalles de como funciona la transmisión, el sistema de frenos o el motor, simplemente puede utilizar el objeto carro completo.

En otras palabras la **abstracción** permite seleccionar características relevantes dentro de un conjunto e identificar comportamientos comunes para definir nuevas entidades. (como hacer una taxonomía).

Una forma de realizar abstracción para proyectos complejos es hacer uso de la clasificación con jerarquía (taxonomía), esto permite en separar la semántica y separarlo en pedazos mas manejables.

Una ventaja de utilizar OO es que los objetos no tienen que exhibir todos sus atributos y comportamientos.

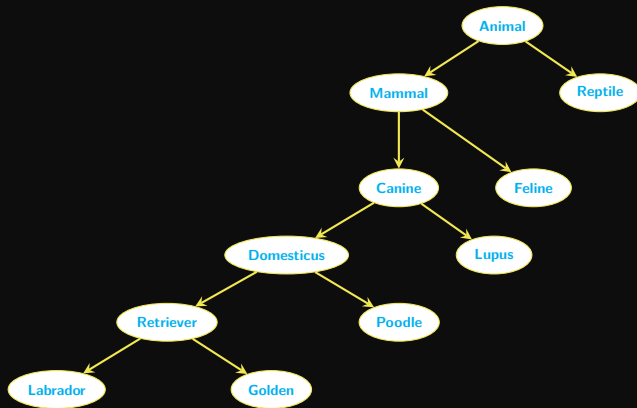
EL encapsular es un mecanismo que une la manipulación de los métodos y de los atributos, ya que permite o prohíbe el acceso (define quien puede y quien no manipularlos)

En un buen diseño una entidad (objeto) solo deberá exhibir las interfaces que otros objetos deben tener para interactuar con ella.

por lo general se definen ya sea métodos (los getters y setters) o interfaces (otras clases) para poder manipular y acceder a los atributos de una clase.

Es el proceso por el cual una clase hereda los atributos y métodos de otra clase, reforzando el árbol de jerarquías de la taxonomía de la abstracción, Promoviendo la reutilización de código.

(dependiendo del lenguaje de programación, algunos lenguajes permiten múltiple herencia), en **JAVA** solo se puede heredar de una clase



Es una de las características mas fuertes de OO y esta fuertemente ligado a la herencia, como su etimología lo define (Múltiples formas), permite que el comportamiento heredado pueda comportarse de diferente manera dependiendo de la naturaleza de quien lo adquiera.

En otras palabras, **permite usar el mismo nombre de métodos para diferentes tipos de comportamiento según el contexto.**

La forma que un objeto de una clase interactúa con otra es a través de relaciones entre ellas. Existen 3 diferentes tipos de relaciones ("Es un", "Tiene un", "Depende de un") en donde la composición es de tipo "Tiene un".

Por ejemplo

la computadora → Tiene un CPU

la computadora → Tiene un GPU

la computadora → Tiene un Teclado

La programación orientado objetos esta fuertemente ligado al desarrollo de software ya que permite que el programa sea más complejos, robustos, extendibles y mantenibles.

El diseño orientado objetos y la programación orientada a objetos son dos conceptos muy diferentes pero ambos se pueden modelar utilizando un lenguaje unificado (UML)

El diseño (arquitectura, modelado etc), es basado en requerimientos y necesidades ya sea de una empresa o de un cliente.

En cuanto la programación(patrones, lenguaje etc) se basa en la implementación de los conceptos de OO para resolver problemas.

Aplicaciones/proyectos a nivel empresarial, las cuales se encargan de mantener la empresa a flote, deben de ser más que un conjunto de módulos de códigos.

Estos proyectos deben de ser estructurados de tal forma que permitan que sean escalables, seguras, robustas, que se ejecuten bajo condiciones de estrés.

También se deben de definir lo suficientemente claro para que el programador pueda rápidamente encontrar y arreglar bugs que puedan suceder mucho después de que los autores originales el proyecto ya no estén.

El modelado es el diseño de una aplicación de software antes de hacer el código.

El modelado es el plano, dentro del desarrollo de software.

Hacer uso del modelado asegura que el proyecto **cumpla con la funcionalidad que el negocio/empresa requiere, cumpla con las necesidades del usuario final sean satisfactorias, también ayuda a la reducción de el costo ya que una aplicación que se implementa sin plan cuesta caro al tratar de extenderla, o que tenga más seguridad.**

UML tiene 13 diagramas que nos ayudan a modelar y diseñar un software los cuales se dividen en 3 categorías principales:

- **Diagramas estructurales (6):** Incluye diagramas de clase, diagramas de objetos, diagramas de componentes, diagrama de estructura de composición, diagrama de paquetes y diagrama de deployment.
- **Diagramas de comportamiento (3):** Incluye diagramas de caso de uso, diagramas de actividad y diagramas de maquinas de estado
- **Diagramas de interacción (4) :** Diagramas de secuencias, diagrama de comunicación, diagrama de tiempos, y diagrama de vista general del tiempo.

Para este curso solamente nos vamos enfocar en

- Diagramas de clases
- Diagramas de objetos

- **JAVA the complete reference**
- **The object oriented thought process**
- **www.uml.org**